

Secure Login System (Mini Project)

Objective:

Develop a secure login web application using Python, Flask, SQLite, and bcrypt for password hashing.

Python Code (app.py):

```
from flask import Flask, render_template, request, redirect, session
import sqlite3, bcrypt

app = Flask(__name__)
app.secret_key = "secret_key"

conn = sqlite3.connect("users.db", check_same_thread=False)
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY,
username TEXT UNIQUE, password BLOB)")
conn.commit()

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method=="POST":
        user=request.form["username"]
        pwd=bcrypt.hashpw(request.form["password"].encode(),
bcrypt.gensalt())
        cur.execute("INSERT INTO users(username,password)
VALUES(?,?)", (user,pwd))
        conn.commit()
        return redirect("/login")
    return "Register Page"

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method=="POST":
        user=request.form["username"]
        pwd=request.form["password"].encode()
        cur.execute("SELECT password FROM users WHERE username=?", (user,))
        row=cur.fetchone()
        if row and bcrypt.checkpw(pwd, row[0]):
            session["user"]=user
            return "Login Successful"
        return "Invalid Credentials"

@app.route("/logout")
def logout():
    session.clear()
    return redirect("/login")

if __name__=="__main__":
    app.run(debug=True)
```

Key Features:

- User registration and login with bcrypt password hashing.
- Parameterized SQL queries to prevent SQL injection.
- Basic session management with logout.

- Input validation can be added before storing user data.
- Optional Two-Factor Authentication (2FA) using OTP libraries.

Expected Outcome:

The application securely stores hashed passwords, prevents SQL injection using parameterized queries, manages user sessions, and supports future integration of 2FA for enhanced security.